Universidad Corporativa Coppel | Programas Académicos

Ingeniería en Desarrollo de Software

Actividad | #2 | Programa Banco Mexicano (parte 1).

Lenguajes de Programación IV.

Ingeniería en Desarrollo de Software

academiaglobal

TUTOR: Aarón Iván Salazar Macías.

ALUMNO: Manuel Marin Sanchez.

FECHA: 26/03/2026.

Indicé.

Portada ……. 1
Indicé …… 2
Introducción ……. 3
Investigación …… 4,5
Desarrollo …….. 6,7,8
Conclusión ……... 9
Referencias ……. 10

Introducción.

La presente actividad académica se desarrolla en el contexto del curso Lenguajes de Programación IV, y tiene como propósito fundamental la creación de un sistema bancario funcional para el Banco Mexicano, utilizando el lenguaje de programación Java 8 bajo el paradigma de Programación Orientada a Objetos (POO). En un mundo cada vez más digitalizado, las instituciones financieras requieren de herramientas tecnológicas que les permitan ofrecer servicios eficientes y seguros a sus clientes, siendo las operaciones básicas como depósitos, retiros y consultas de saldo las más demandadas en el día a día.

El programa desarrollado en esta primera etapa consiste en una aplicación de consola que presenta un menú principal con cuatro opciones esenciales: depósito, retiro, consulta de saldo y salida del sistema. Cada una de estas operaciones ha sido implementada siguiendo estrictamente los principios de la Programación Orientada a Objetos, particularmente el encapsulamiento, mediante la creación de una clase denominada CuentaBancaria, la cual contiene los atributos privados que representan los datos sensibles de la cuenta, así como los métodos públicos que permiten manipular dichos datos de manera controlada y segura.

La implementación del módulo de depósitos presenta una característica particularmente importante: la capacidad de realizar múltiples depósitos de forma consecutiva, permitiendo al usuario decidir si desea continuar o regresar al menú principal. Por su parte, el módulo de retiros incorpora validaciones que verifican la disponibilidad de saldo suficiente antes de autorizar la operación, evitando así que el cliente pueda sobregirar su cuenta. Estas funcionalidades, aunque básicas, representan la base fundamental sobre la cual se construirán características más avanzadas en etapas posteriores del proyecto, demostrando la importancia de una arquitectura de software bien diseñada desde sus cimientos.

Investigación.
Como declarar una clase en lenguaje Java:

```
public class Persona {
    // Atributos
    private String nombre;
    private int edad;

    // Constructor
    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    // Métodos
    public void saludar() {
        System.out.println("Hola, soy " + nombre);
    }

    // Getters y Setters
    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }
}
```

Como declarar una clase en lenguaje Python:

```
class Persona:
    # Constructor
    def __init__(self, nombre, edad):
        self._nombre = nombre  # Atributo "privado" por convención
        self._edad = edad

    # Métodos
    def saludar(self):
        print(f"Hola, soy {self._nombre}")

    # Getters y Setters (usando propiedades)
    @property
    def nombre(self):
        return self._nombre

    @nombre.setter
    def nombre(self, valor):
        self._nombre = valor
```

Como declarar una clase en lenguaje C#:

```
public class Persona
{
    // Atributos (propiedades automáticas)
    public string Nombre { get; set; }
    public int Edad { get; set; }

    // Constructor
    public Persona(string nombre, int edad)
    {
        Nombre = nombre;
        Edad = edad;
    }

    // Método
    public void Saludar()
    {
        Console.WriteLine($"Hola, soy {Nombre}");
    }
}
```

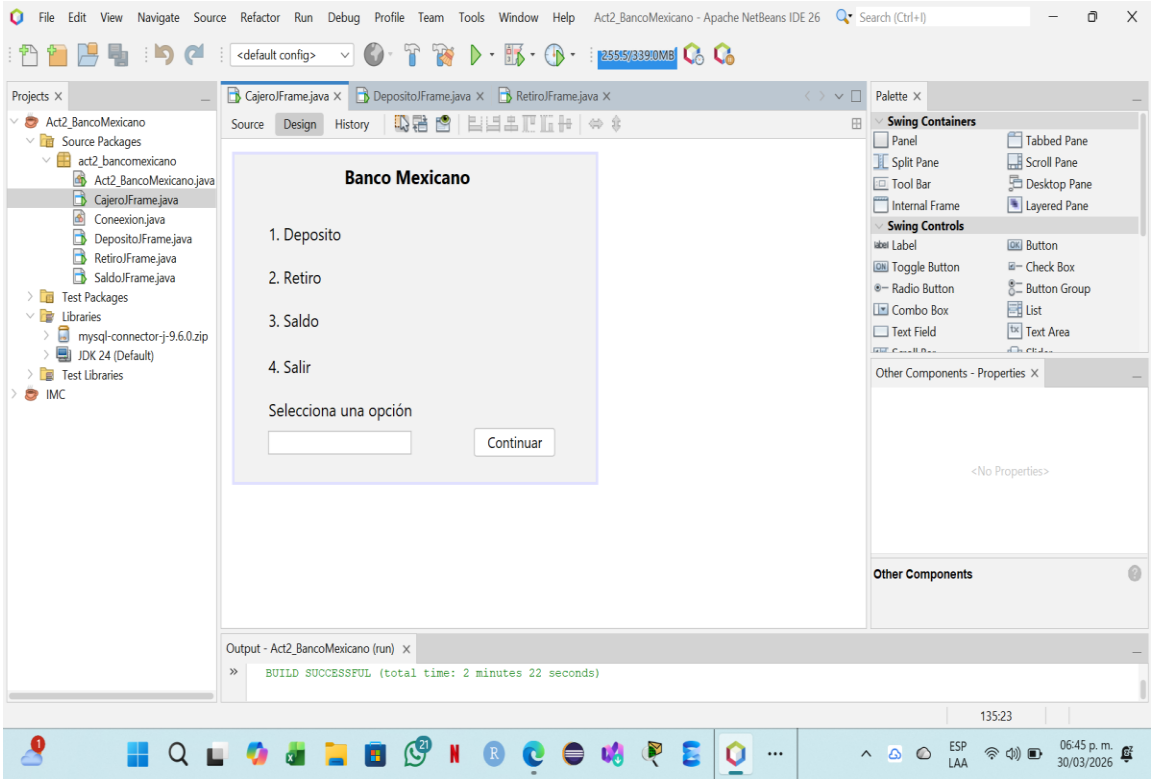
Como declarar una clase en lenguaje C++:

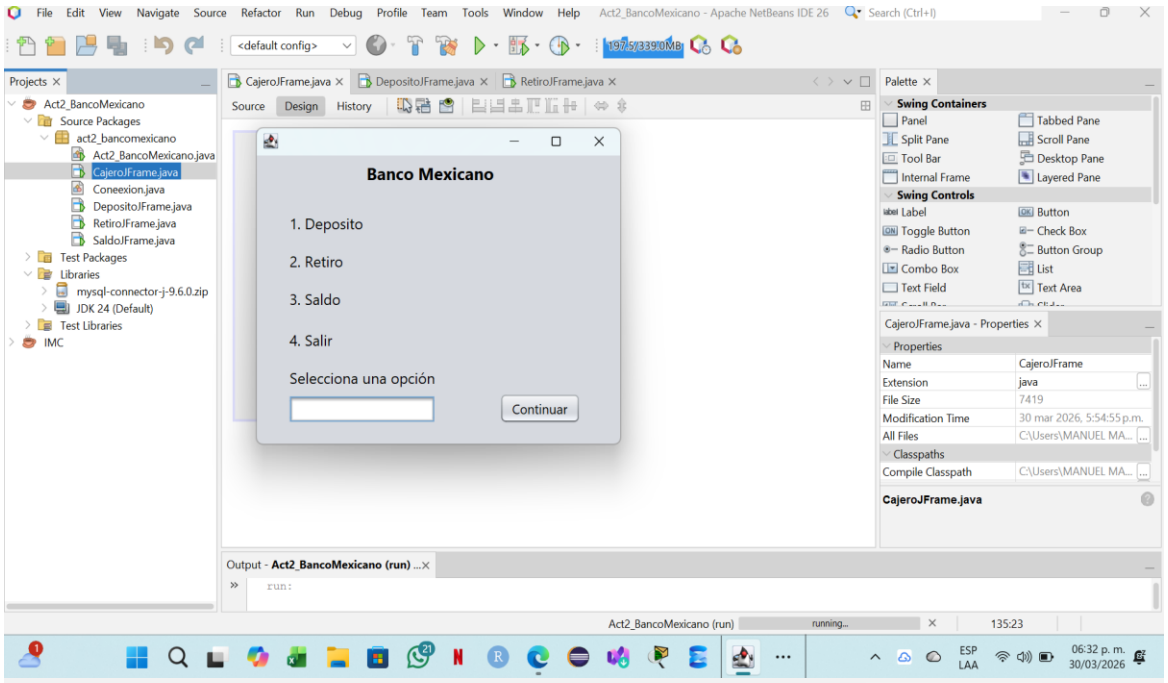
```
#include <iostream>
#include <string>
using namespace std;

class Persona {
private:
    // Atributos privados
    string nombre;
    int edad;

public:
    // Constructor
    Persona(string nombre, int edad) {
        this->nombre = nombre;
        this->edad = edad;
    }

    // Método
    void saludar() {
        cout << "Hola, soy " << nombre << endl;
    }

    // Getters y Setters
    string getNombre() { return nombre; }
    void setNombre(string nombre) { this->nombre = nombre; }
};
```

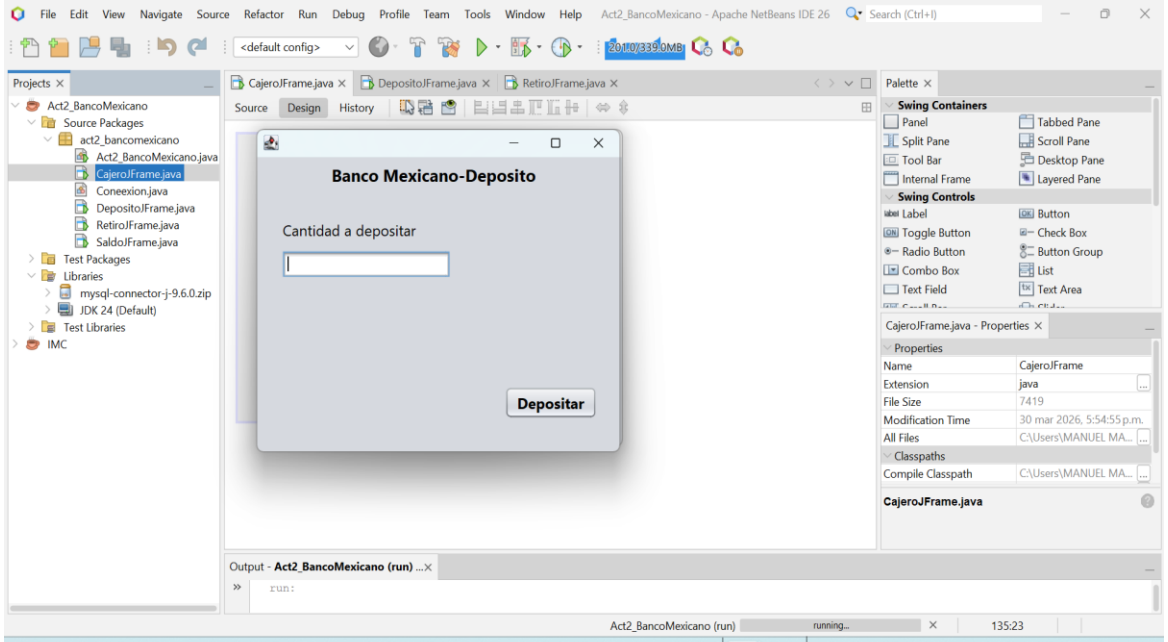
Como declarar una clase en lenguaje PHP:

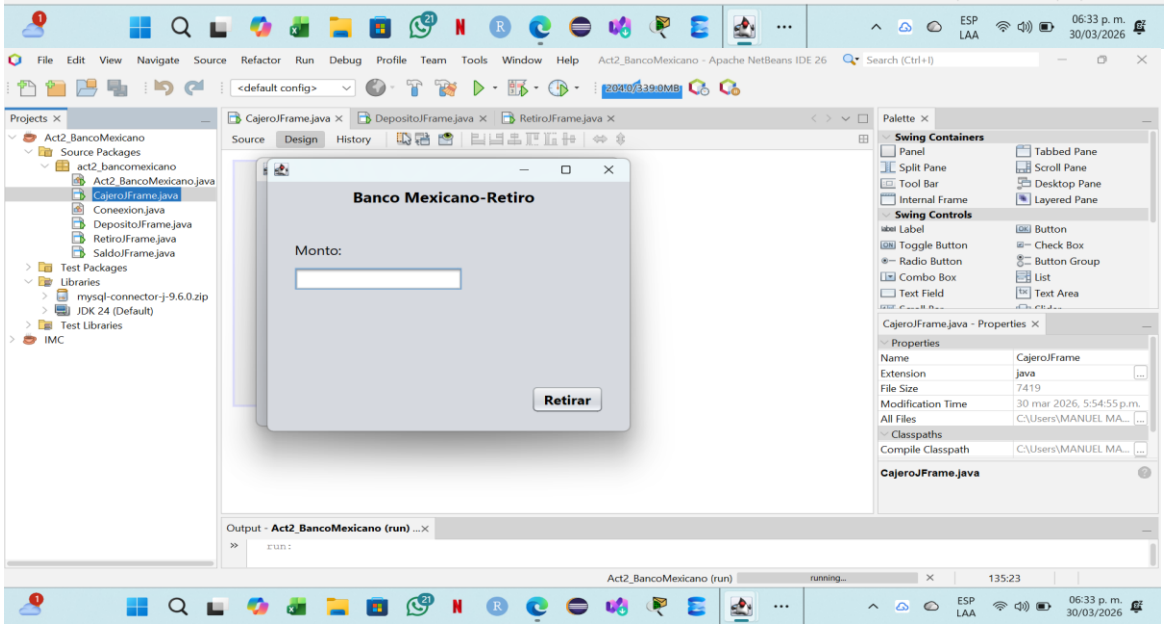
```php
<?php
class Persona {
    // Atributos
    private $nombre;
    private $edad;

    // Constructor
    public function __construct($nombre, $edad) {
        $this->nombre = $nombre;
        $this->edad = $edad;
    }

    // Método
    public function saludar() {
        echo "Hola, soy " . $this->nombre;
    }

    // Getters y Setters
    public function getNombre() { return $this->nombre; }
    public function setNombre($nombre) { $this->nombre = $nombre; }
}
?>
```

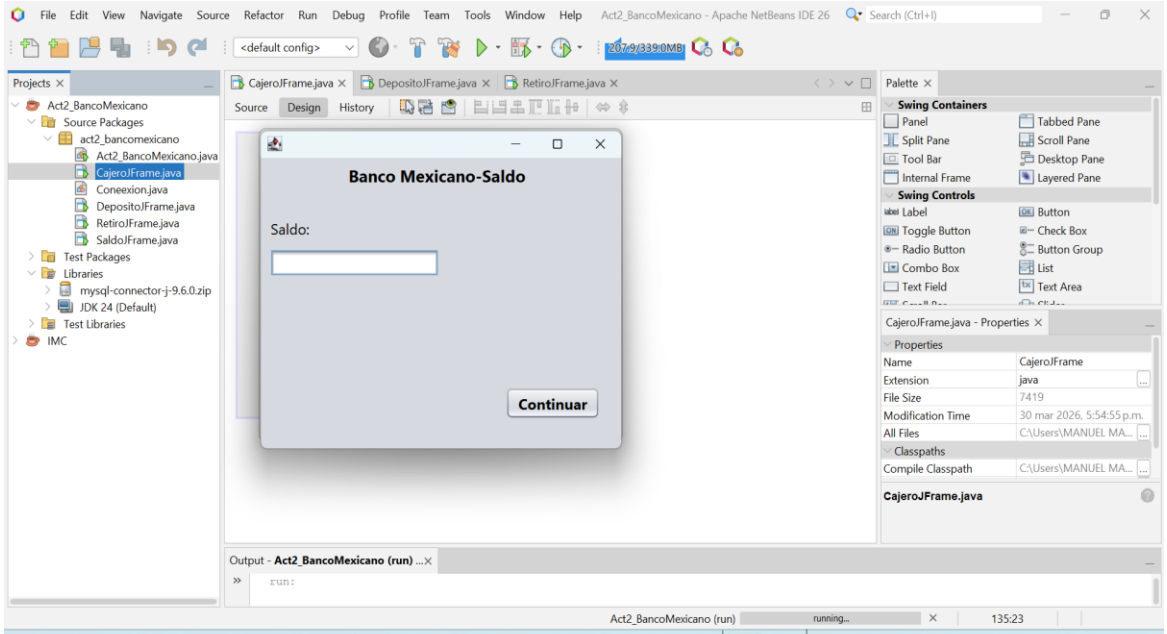
Desarrollo:

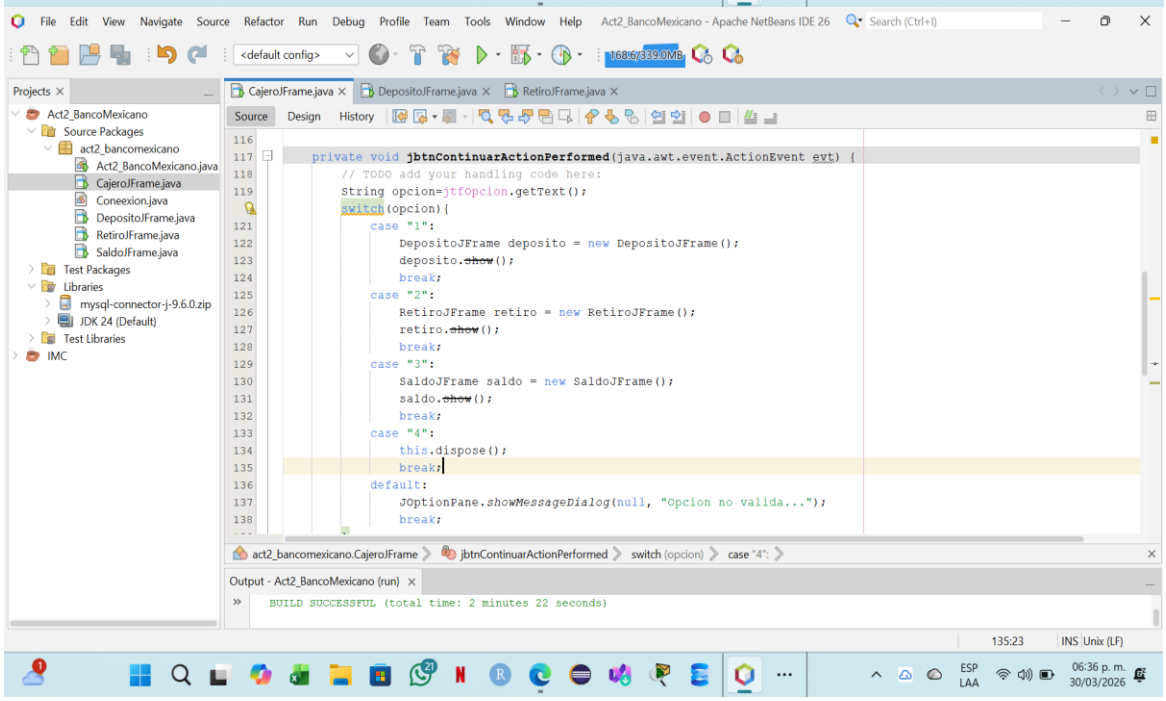



Banco Mexicano
1. Deposito
2. Retiro
3. Saldo
4. Salir
Selecciona una opción
Continuar


Banco Mexicano-Deposito
Cantidad a depositar
Depositar


Banco Mexicano-Retiro
Monto:
Retirar





https://github.com/ManuelMarin14/Manuel_Marin_Act2.git

Conclusiones.

El desarrollo de la primera parte del programa Banco Mexicano ha representado un ejercicio integral que permitió aplicar en un contexto práctico los conceptos fundamentales de la Programación Orientada a Objetos estudiados a lo largo del curso.

Uno de los aprendizajes más significativos fue la implementación de la lógica de múltiples depósitos consecutivos, que requirió el uso de estructuras de control como bucles while y condicionales if-else, así como la correcta gestión de la entrada de datos por teclado mediante la clase Scanner. Esta funcionalidad, aunque aparentemente simple, enseña la importancia de diseñar interfaces de usuario intuitivas que permitan al usuario realizar tareas repetitivas sin tener que navegar constantemente por los menús.

La validación de saldo en el módulo de retiros resultó particularmente relevante, ya que ilustra cómo un programa debe anticiparse a posibles errores del usuario y protegerlo de acciones que podrían tener consecuencias no deseadas, como intentar retirar más dinero del disponible. Este tipo de validaciones son esenciales en el desarrollo de software financiero, donde la precisión y la seguridad son prioritarias.

En conclusión, esta actividad no solo cumplió con los requisitos funcionales establecidos, sino que también sentó las bases sólidas para la siguiente etapa del proyecto, donde se implementarán funcionalidades más complejas como el registro histórico de transacciones, la persistencia de datos y una interfaz gráfica más amigable. La experiencia adquirida en el manejo de estructuras de control, clases y objetos será invaluable para enfrentar los desafíos venideros en el desarrollo de software profesional.

Referencias.

1. Cosmina, L. (2018). *Java for Absolute Beginners: learn to program the fundamentals the Java 9+ ways*. Apress.
2. Deitel, P. J., & Deitel, H. M. (2016). *Java How to Program, Early Objects* (10th ed.). Pearson.
3. Oracle Corporation. (2023). *The Java™ Tutorials*. https://docs.oracle.com/javase/tutorial/
4. Schildt, H. (2018). *Java: The Complete Reference* (11th ed.). McGraw-Hill Education.
5. Sierra, K., & Bates, B. (2020). *OCA/OCP Java SE 8 Programmer Certification Kit*. John Wiley & Sons.